

FlotaOS

Estructuras de Base de Datos

Modelo relacional vinculado a cada ventana del sistema

App Driver

App Técnico

Pagos Campo

Web GPS

Web Mantenimiento

Web KPIs

Caja Chica

Roles

Supabase PostgreSQL · Row Level Security · 24 tablas

Mapa Global — 24 Tablas del Sistema

Todas las tablas organizadas por módulo, con sus relaciones principales

CORE vehicles drivers providers staff	M1 · GPS gps_tracks geofences geofence_events checklist_submissions	M2 · MANTENIMIENTO catalogo_mantenimiento plan_mantenimiento maintenance_records ordenes_trabajo spare_parts	CAJA CHICA petty_cash_transactions petty_cash_accounts reimbursement_batches petty_cash_ledger
M3 · COMBUSTIBLE fuel_records fuel_anomalies idle_records fuel_cards	M4 · CONDUCTORES driving_events driver_scores fatigue_records driver_rewards	M5 · DOCUMENTOS fleet_documents document_alerts rental_contracts	M6 · KPIs kpi_snapshots fleet_summaries reports

vehicles & drivers son las tablas centrales — referenciadas por todos los módulos

vehicles · drivers · providers · staff

vehicles		
PK	id	uuid PK
	plate_number	varchar(8) UNIQUE
	brand	varchar(50)
	model	varchar(100)
	year	smallint
	vehicle_type	enum (own rented)
	status	enum
	fuel_type	enum
	current_km	integer
	horometro_actual	integer
FK	provider_id	uuid
	created_at	timestampz

drivers		
PK	id	uuid PK
FK	user_id	uuid
	full_name	varchar(200)
	dni	varchar(8) UNIQUE
	license_number	varchar(20)
	license_expiry	date
	license_category	varchar(5)
	medical_cert_expiry	date
	score	smallint (0-100)
FK	assigned_vehicle_id	uuid
	is_active	boolean

providers		
PK	id	uuid PK
	company_name	varchar(200)
	ruc	varchar(11) UNIQUE
	contact_name	varchar(100)
	contact_phone	varchar(20)
	contact_email	varchar(100)
	is_active	boolean
	created_at	timestampz

staff		
PK	id	uuid PK
FK	user_id	uuid
	full_name	varchar(200)
	email	varchar(100)
FK	region_id	uuid
	role	enum
FK	team_id	uuid
	is_active	boolean



Row Level Security (RLS) habilitado en TODAS las tablas

vehicles: SELECT para authenticated · INSERT/UPDATE/DELETE solo para admin y fleet_manager
drivers: SELECT para authenticated · drivers pueden leer su propio perfil (user_id = auth.uid())

gps_tracks · geofences · geofence_events · checklist_submissions · trip_events

gps_tracks (particionada por mes)		
PK	id	bigserial PK
FK	vehicle_id	uuid
latitude		decimal(10,7)
longitude		decimal(10,7)
speed		smallint (km/h)
heading		smallint (0-360°)
altitude		smallint
engine_on		boolean
recorded_at		timestamptz NOT NULL

trip_events		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
FK	trip_id	uuid
event_type		enum
latitude		decimal(10,7)
longitude		decimal(10,7)
photo_url		varchar(500)
notes		text
is_force_majeure		boolean
fm_type		enum (nullable)
fm_photos		jsonb (3 urls)
recorded_at		timestamptz

trips		
PK	id	uuid PK
FK	team_id	uuid
FK	vehicle_id	uuid
FK	driver_id	uuid
origin		varchar(200)
destination		varchar(200)
node_code		varchar(50)
trip_date		date
status		enum
started_at		timestamptz
completed_at		timestamptz
total_km		decimal(8,2)

geofences		
PK id	uuid PK	
name	varchar(100)	
coordinates	jsonb [[lng,lat]]	
alert_on_entry	boolean	
alert_on_exit	boolean	
is_active	boolean	

checklist_submissions		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
FK	trip_id	uuid
items_json		jsonb (item+status+foto)
has_issues		boolean
vehicle_blocked		boolean
submitted_at		timestamptz

geofence_events		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	geofence_id	uuid
event_type		enum (entry exit)
latitude		decimal(10,7)
longitude		decimal(10,7)
occurred_at		timestamptz
is_resolved		boolean

event_type ENUM (trip_events)

llegada_sitio · parada · reinicio · llegada_destino

fm_type ENUM (fuerza mayor)

derrumbe · inundacion · caida_puente · lluvias · contingencia_social · otros

PARTICIONAMIENTO gps_tracks

Tabla particionada por mes (PARTITION BY RANGE recorded_at). Índice compuesto: vehicle_id + recorded_at DESC para consultas eficientes.

Regla de negocio — Fuerza Mayor

is_force_majeure=true requiere fm_photos con exactamente 3 URLs. Constraint CHECK en BD. Sin las 3 fotos el INSERT es rechazado.

teams · team_members · tasks · task_events · nodes

teams		
PK	id	<i>uuid PK</i>
	name	<i>varchar(100)</i>
FK	region_id	<i>uuid</i>
FK	zone_id	<i>uuid</i>
FK	vehicle_id	<i>uuid</i>
FK	supervisor_id	<i>uuid</i>
	trip_date	<i>date</i>
	is_active	<i>boolean</i>
	created_at	<i>timestampz</i>

team_members		
PK	id	<i>uuid PK</i>
FK	team_id	<i>uuid</i>
FK	staff_id	<i>uuid</i>
	role	<i>enum (driver tecnico)</i>
	subgroup	<i>smallint (1 2 3)</i>
	node_assigned	<i>varchar(50)</i>
	joined_at	<i>timestampz</i>
	left_at	<i>timestampz (nullable)</i>

tasks		
PK	id	<i>uuid PK</i>
FK	team_id	<i>uuid</i>
FK	assigned_to	<i>uuid</i>
FK	node_id	<i>uuid</i>
	task_type	<i>enum</i>
	title	<i>varchar(200)</i>
	description	<i>text</i>
	priority	<i>enum (low med high)</i>
	status	<i>enum</i>
	due_date	<i>date</i>
	completed_at	<i>timestampz</i>

nodes		
PK	id	<i>uuid PK</i>
	code	<i>varchar(50) UNIQUE</i>
	name	<i>varchar(200)</i>
FK	region_id	<i>uuid</i>
FK	zone_id	<i>uuid</i>
	latitude	<i>decimal(10,7)</i>
	longitude	<i>decimal(10,7)</i>
	node_type	<i>enum</i>
	has_generator	<i>boolean</i>
	is_active	<i>boolean</i>

task_events		
PK	id	<i>uuid PK</i>
FK	task_id	<i>uuid</i>
FK	team_id	<i>uuid</i>
FK	staff_id	<i>uuid</i>
	event_type	<i>enum</i>
	latitude	<i>decimal(10,7)</i>
	longitude	<i>decimal(10,7)</i>
	photo_urls	<i>jsonb []</i>
	notes	<i>text</i>
	is_force_majeure	<i>boolean</i>
	fm_type	<i>enum (nullable)</i>
	recorded_at	<i>timestampz</i>

zones		
PK	id	<i>uuid PK</i>
	name	<i>varchar(100)</i>
FK	region_id	<i>uuid</i>
FK	supervisor_id	<i>uuid</i>
	is_active	<i>boolean</i>

regions		
PK	id	<i>uuid PK</i>
	name	<i>varchar(100)</i>
	code	<i>varchar(10) UNIQUE</i>
FK	coordinator_id	<i>uuid</i>
	color_hex	<i>varchar(6)</i>
	is_active	<i>boolean</i>

task_type ENUM

instalacion · mantenimiento_preventivo · mantenimiento_correctivo · carga_generador · inspeccion · otro

Subgrupos de técnicos

subgroup en team_members permite que el driver deje técnicos en nodos distintos. Cada subgroup registra task_events independientemente, todos vinculados al mismo team_id.

petty_cash_transactions · petty_cash_accounts · reimbursement_batches · staff_bank_accounts · petty_cash_ledger

petty_cash_transactions	
PK id	uuid PK
seq_number	integer (por región)
FK region_id	uuid
FK staff_id	uuid
FK trip_id	uuid
transaction_date	date
concept	enum
expense_subtype	enum
description	text
node_code	varchar(50)
vehicle_plate	varchar(8)
amount	decimal(10,2)
status	enum
voucher_type	enum
voucher_number	varchar(50)
photo_url	varchar(500)
reimbursement_code	varchar(100)
concept ENUM vehicle_fuel · dg_refuel · peajes · viaticos · consumibles · vehicle_expenses · operation_expense · boat_expense · otros	
observations	text
FK batch_id	uuid
created_at	timestamptz

petty_cash_accounts	
PK id	uuid PK
FK region_id	uuid
permanent_fund	decimal(10,2)
current_balance	decimal(10,2)
amount_in_field	decimal(10,2)
to_be_submitted	decimal(10,2)
in_reimbursement	decimal(10,2)
gap	decimal(10,2)
period_start	date
is_active	boolean

petty_cash_ledger	
PK id	uuid PK
FK staff_id	uuid
FK region_id	uuid
ledger_date	date
total_disbursed	decimal(10,2)
total_rendered	decimal(10,2)
in_field	decimal(10,2)
to_submit	decimal(10,2)
in_reimbursement_process	decimal(10,2)
gap	decimal(10,2)

reimbursement_batches	
PK id	uuid PK
batch_code	varchar(100) UNIQUE
FK region_id	uuid
FK staff_id	uuid
batch_type	enum
total_amount	decimal(10,2)
transaction_count	integer
status	enum
submitted_at	date
paid_at	date (nullable)
FK approved_by	uuid
payment_reference	varchar(100)

status ENUM
rendido · pendiente · pendiente_carga_f · en_reembolso · inhouse · expense_record · observado · depositado

staff_bank_accounts	
PK id	uuid PK
FK staff_id	uuid
bank_name	varchar(100)
account_number	varchar(30)
cci	varchar(20)
is_primary	boolean
is_active	boolean

Reconciliación automática
expense_subtype = fuel_pickup → genera fuel_record en M3. expense_subtype = dg_refuel → vincula con nodo generador. Sin intervención manual.

gps_tracks · geofences · geofence_events · alerts · trips · trip_events

alerts (tabla transversal)		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
FK	team_id	uuid
alert_type		enum
title		varchar(200)
description		text
severity		enum (low med high critical)
latitude		decimal(10,7)
longitude		decimal(10,7)
is_resolved		boolean
resolved_at		timestamptz
FK	resolved_by	uuid
created_at		timestamptz

driving_events		
PK	id	uuid PK
FK	driver_id	uuid
FK	vehicle_id	uuid
event_type		enum
severity		enum (low med high)
speed		smallint
latitude		decimal(10,7)
longitude		decimal(10,7)
recorded_at		timestamptz

idle_records		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
duration_minutes		integer
liters_wasted		decimal(6,2)
cost_wasted		decimal(8,2)
latitude		decimal(10,7)
longitude		decimal(10,7)
started_at		timestamptz
ended_at		timestamptz

alert_type ENUM

geocerca_entrada · geocerca_salida · exceso_velocidad ·
ralenti_prolongado · zona_restringida · bateria_baja · sin_señal ·
fuerza_mayor · doc_vencimiento · mantenimiento_vencido

```
-- Vista GPS en tiempo real para el mapa web
-- Última posición de cada vehículo activo
SELECT DISTINCT ON (g.vehicle_id)
  g.vehicle_id,
  v.plate_number,
  v.status,
  g.latitude,
  g.longitude,
  g.speed,
  g.engine_on,
  g.recorded_at,
  d.full_name AS driver_name
FROM gps_tracks g
JOIN vehicles v ON v.id = g.vehicle_id
LEFT JOIN drivers d ON d.assigned_vehicle_id = g.vehicle_id
WHERE v.status = 'active'
  AND g.recorded_at > NOW() - INTERVAL '10 minutes'
ORDER BY g.vehicle_id, g.recorded_at DESC;
```

```
-- Alertas activas para el dashboard del supervisor
-- Filtrado por supervisor_id (RLS policy)
SELECT
  a.id,
  a.alert_type,
  a.severity,
  a.title,
  a.created_at,
  v.plate_number,
  d.full_name AS driver
FROM alerts a
LEFT JOIN vehicles v ON v.id = a.vehicle_id
LEFT JOIN drivers d ON d.id = a.driver_id
WHERE a.is_resolved = false
  AND a.severity IN ('high','critical')
ORDER BY a.created_at DESC
LIMIT 20;
```

catalogo_mantenimiento · plan_mantenimiento · maintenance_records · ordenes_trabajo · spare_parts · maintenance_predictions

catalogo_mantenimiento		
PK	id	uuid PK
	nombre_elemento	varchar(100)
	unidad_medida	enum (km horas meses)
	descripcion	text
	procedimiento_sop	text
	is_active	boolean

plan_mantenimiento		
PK	id	uuid PK
FK	vehiculo_id	uuid
FK	elemento_id	uuid
	frecuencia_valor	integer
	frecuencia_unidad	enum (km horas meses)
	alerta_amarilla	integer (km días)
	alerta_roja	integer (km días)
	is_active	boolean

maintenance_records		
PK	id	uuid PK
FK	vehiculo_id	uuid
FK	elemento_id	uuid
FK	orden_trabajo_id	uuid
	fecha_servicio	date
	lectura_odometro	integer
	lectura_horometro	integer
	costo_repuestos	decimal(10,2)
	costo_mano_obra	decimal(10,2)
	costo_total	decimal(10,2)
	taller_proveedor	varchar(100)
	tipo_servicio	enum
	observaciones	text

ordenes_trabajo		
PK	id	uuid PK
FK	vehiculo_id	uuid
	numero_ot	varchar(20) UNIQUE
	fecha_programada	date
	fecha_completada	date
	status	enum
	urgencia	enum
	taller	varchar(100)
	costo_estimado	decimal(10,2)
	costo_real	decimal(10,2)
	trabajos	jsonb []
	repuestos	jsonb []
FK	generado_por	uuid

spare_parts		
PK	id	uuid PK
	part_number	varchar(50)
	name	varchar(200)
	stock_qty	integer
	min_stock	integer
	unit_cost	decimal(8,2)
	supplier	varchar(100)
	is_active	boolean

maintenance_predictions		
PK	id	uuid PK
FK	vehiculo_id	uuid
	componente	varchar(100)
	nivel_riesgo	enum (low med high critical)
	km_estimado_falla	integer
	costo_preventivo	decimal(10,2)
	costo_correctivo	decimal(10,2)
	resumen_ia	text (Claude API)
	analizado_at	timestamptz

alertas_mantenimiento		
PK	id	uuid PK
FK	vehiculo_id	uuid
FK	plan_id	uuid
	tipo_alerta	enum
	urgencia	enum
	km_vencimiento	integer
	fecha_vencimiento	date
	notificacion_enviada	boolean
	is_resolved	boolean
	created_at	timestamptz

Leak Detector — SQL

SELECT vehiculo_id, SUM(CASE tipo='preventive' THEN costo_total END) prev, SUM(CASE tipo='corrective' THEN costo_total END) corr FROM maintenance_records GROUP BY vehiculo_id

kpi_snapshots · fleet_summaries · fuel_records · fuel_anomalies · fuel_cards · driver_scores · fatigue_records

kpi_snapshots		
PK	id	uuid PK
	snapshot_date	date
FK	vehicle_id	uuid (nullable)
	cost_per_km	decimal(8,4)
	km_per_liter	decimal(6,3)
	mechanical_availability	decimal(5,2) %
	mtbf_days	decimal(6,2)
	mttr_hours	decimal(6,2)
	speeding_events	integer
	idle_hours	decimal(6,2)
	fuel_cost	decimal(10,2)
	maintenance_cost	decimal(10,2)
UNIQUE	(snapshot_date, vehicle_id)	

fleet_summaries		
PK	id	uuid PK
	summary_date	date
	period_type	enum (daily/weekly/monthly)
FK	region_id	uuid
	total_vehicles_active	integer
	total_km	decimal(10,2)
	total_fuel_liters	decimal(8,2)
	total_fuel_cost	decimal(10,2)
	total_maint_cost	decimal(10,2)
	avg_cost_per_km	decimal(8,4)
	avg_km_per_liter	decimal(6,3)
	fleet_availability	decimal(5,2) %
	ai_savings_estimated	decimal(10,2)

fuel_records		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
FK	trip_id	uuid
	liters	decimal(6,2)
	total_cost	decimal(8,2)
	km_at_refuel	integer
	fuel_card_id	varchar(20)
	station_name	varchar(100)
	receipt_photo	varchar(500)
	is_anomaly	boolean
	anomaly_type	varchar(30)
	recorded_at	timestampz

fuel_anomalies		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	driver_id	uuid
	anomaly_type	enum
	description	text
	severity	enum
	estimated_loss	decimal(8,2)
	claude_analysis	text
	is_resolved	boolean
	detected_at	timestampz

driver_scores		
PK	id	uuid PK
FK	driver_id	uuid
	score_date	date
	total_score	smallint (0-100)
	harsh_braking_count	integer
	harsh_accel_count	integer
	speeding_count	integer
	idle_minutes	integer
	km_per_liter	decimal(6,3)

fatigue_records		
PK	id	uuid PK
FK	driver_id	uuid
	work_date	date
	hours_driven	decimal(4,2)
	breaks_taken	smallint
	limit_reached	boolean
	last_break_at	timestampz

```
-- Cron nocturno: calcular KPIs del día (Supabase Edge Function)
INSERT INTO kpi_snapshots (snapshot_date, vehicle_id,
  cost_per_km, km_per_liter, mechanical_availability)
SELECT
  CURRENT_DATE,
  v.id,
  ROUND((SUM(fr.total_cost) + SUM(mr.costo_total)) /
    NULLIF(SUM(gt.km_recorridos), 0), 4),
  ROUND(SUM(fr.liters) /
    NULLIF(SUM(gt.km_recorridos), 0), 3),
  ROUND(100.0 * (1 - SUM(downtime_hours) / 24), 2)
FROM vehicles v
LEFT JOIN fuel_records fr ON fr.vehicle_id = v.id
  AND fr.recorded_at::date = CURRENT_DATE
LEFT JOIN maintenance_records mr ON mr.vehiculo_id = v.id
  AND mr.fecha_servicio = CURRENT_DATE
GROUP BY v.id
ON CONFLICT (snapshot_date, vehicle_id) DO UPDATE
  SET cost_per_km = EXCLUDED.cost_per_km;
```

fleet_documents · document_alerts · rental_contracts · driver_rewards

fleet_documents		
PK	id	uuid PK
FK	vehicle_id	uuid (nullable)
FK	driver_id	uuid (nullable)
FK	provider_id	uuid (nullable)
	document_type	enum
	expiry_date	date NOT NULL
	file_path	varchar(500)
	is_expired	boolean (calculado)
	days_until_expiry	integer (calculado)
FK	uploaded_by	uuid
	notes	text
	created_at	timestampz

document_alerts		
PK	id	uuid PK
FK	document_id	uuid
	days_before_expiry	integer (30 15 7 0)
	notification_sent	boolean
	sent_at	timestampz
	notified_to	jsonb [emails]
	is_resolved	boolean

rental_contracts		
PK	id	uuid PK
FK	vehicle_id	uuid
FK	provider_id	uuid
	contract_number	varchar(50)
	start_date	date
	end_date	date
	monthly_cost	decimal(10,2)
	km_limit_monthly	integer
	conditions	text
	status	enum (active expired cancelled)
	inspection_in	jsonb (fotos)
	inspection_out	jsonb (fotos)
	file_path	varchar(500)

driver_rewards		
PK	id	uuid PK
FK	driver_id	uuid
	reward_type	enum
	description	varchar(200)
	awarded_date	date
	period	varchar(20) (YYYY-WW)
	is_claimed	boolean
	claimed_at	timestampz

document_type ENUM

soat · technical_revision · circulation_permit · insurance · rental_contract · driver_license · medical_cert · others

```
-- Cron diario: generar alertas de vencimiento (00:00 UTC)
INSERT INTO document_alerts (document_id, days_before_expiry)
SELECT
    fd.id,
    (fd.expiry_date - CURRENT_DATE)::integer AS days_left
FROM fleet_documents fd
WHERE
    fd.expiry_date - CURRENT_DATE IN (30, 15, 7, 3, 0)
    AND NOT EXISTS (
        SELECT 1 FROM document_alerts da
        WHERE da.document_id = fd.id
            AND da.days_before_expiry =
                (fd.expiry_date - CURRENT_DATE)::integer
            AND da.notification_sent = true
    );

-- Si days_left <= 0: actualizar vehicle status a 'out_of_service'
UPDATE vehicles SET status = 'out_of_service'
WHERE id IN (
    SELECT vehicle_id FROM fleet_documents
    WHERE document_type = 'soat'
        AND expiry_date < CURRENT_DATE
);
```

reward_type ENUM

mejor_conductor_semana · mejor_km_l · cero_eventos_mes · puntualidad · reconocimiento_especial

Storage — bucket fleet-docs

Bucket PRIVADO. URLs firmadas con expiración 1h máximo. Nunca exponer path directo. Documentos organizados: /region/vehicle_id/doc_type/fecha.pdf

Row Level Security — Políticas por rol en cada tabla

ROL	vehicles	drivers	teams	tasks	gps	fuel	maint	docs	caja	KPIs
Driver (app)	👁 propio	👁 propio	👁 propio	👁 asig.	👁 propio	✎ crear	👁 propio	👁 propio	✎ crear	—
Técnico (app)	👁 propio	👁 propio	👁 propio	✎ crear	👁 propio	✎ crear	👁 propio	—	✎ crear	—
Supervisor zona	👁 zona	👁 zona	👁 zona	✎ zona	👁 zona	👁 zona	👁 zona	👁 zona	✎ zona	👁 zona
Coordinador reg.	👁 región	👁 región	✎ región	✎ región	👁 región	👁 región	✎ región	✎ región	✎ región	👁 región
Asistente reg.	👁 región	👁 región	👁 región	👁 región	👁 región	👁 región	👁 región	✎ región	✎ región	👁 región
Director OSC	👁 todo	👁 todo	✎ todo	✎ todo	👁 todo	👁 todo	✎ todo	✎ todo	✎ todo	✎ todo

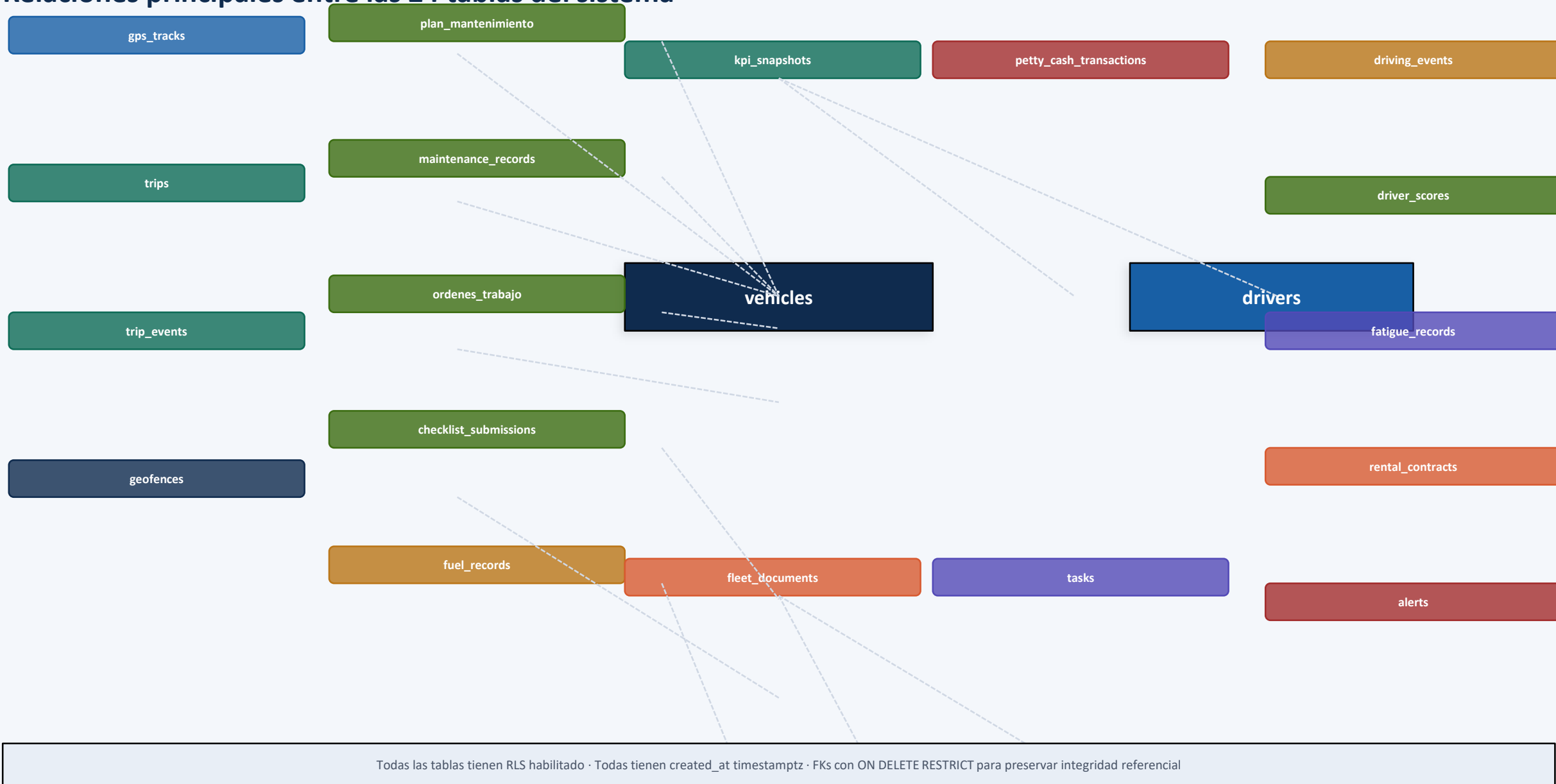
```
-- RLS Policy: Supervisor ve solo sus teams
CREATE POLICY "supervisor_sees_own_teams"
ON trips FOR SELECT
USING (
  team_id IN (
    SELECT t.id FROM teams t
    JOIN zones z ON z.id = t.zone_id
    WHERE z.supervisor_id = auth.uid()
  )
);

-- RLS Policy: Driver ve solo su vehículo asignado
CREATE POLICY "driver_sees_own_vehicle"
ON gps_tracks FOR SELECT
USING (
  vehicle_id IN (
    SELECT assigned_vehicle_id FROM drivers
    WHERE user_id = auth.uid()
  )
);
```

```
-- RLS Policy: Coordinador ve toda su región
CREATE POLICY "coordinator_sees_region"
ON petty_cash_transactions FOR ALL
USING (
  region_id IN (
    SELECT id FROM regions
    WHERE coordinator_id = auth.uid()
    OR id IN (
      SELECT region_id FROM staff
      WHERE user_id = auth.uid()
      AND role IN ('coordinador','asistente')
    )
  )
);

-- RLS Policy: Técnico crea transacciones de campo
CREATE POLICY "tecnico_creates_transactions"
ON petty_cash_transactions FOR INSERT
WITH CHECK (
  staff_id = (SELECT id FROM staff WHERE user_id = auth.uid())
  AND created_at::date = CURRENT_DATE
);
```

Relaciones principales entre las 24 tablas del sistema



Convenciones de la Base de Datos



Claves primarias

Siempre UUID generado con `gen_random_uuid()`. Nunca enteros autoincrementales para tablas de negocio.



Row Level Security

Habilitado en las 24 tablas sin excepción. Políticas definidas por rol de Supabase Auth JWT.



Timestamps

`created_at` TIMESTAMPTZ DEFAULT NOW() en todas las tablas. `updated_at` con trigger automático en tablas mutables.



Nomenclatura

`snake_case` para tablas y columnas. Sin prefijos de tabla. Enums en inglés lowercase con underscore.



Particionamiento

`gps_tracks` particionada por mes (RANGE en `recorded_at`). Retención configurable por política de datos.



IA — Claude API

Resultados de análisis predictivo guardados en `maintenance_predictions` y `fuel_anomalies` con timestamp. Nunca se consulta la API en tiempo real desde queries SQL.



Supabase Realtime

Canal `gps_tracks` para WebSocket del mapa. Canal `alerts` para notificaciones push. Filtrado por `vehicle_id` o `team_id`.



Storage

Bucket `fleet-docs` (privado). URLs firmadas máx. 1h. Estructura: `/region/entity_id/doc_type/YYYYMMDD.ext`